

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Роменский М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Расстоянием Левенштейна называется минимальная суммарная стоимость операций вставки одного символа, удаления одного символа и замены одного символа на другой, которые переводят одну строку в другую.

В базовой части работы требуется реализовать вычисление расстояния Левенштейна между двумя строками с заданными стоимостями операций.

В **варианте 10** после выполнения базовых вычислений пользователь выбирает одну из строк (первую или вторую) и вводит её продолжение. Требуется вычислить новое редакционное расстояние с учётом изменившейся строки, **используя уже вычисленную ранее таблицу динамического программирования**. Ключевое требование: временная сложность пересчёта должна зависеть только от длины добавления и длины другой (не расширяемой) строки, и **не должна зависеть от изначальной длины расширенной строки**.

Для всех вариантов действуют общие ограничения:

- Для стоимостей операций выполняется правило треугольника: если две последовательные операции можно заменить одной, то суммарная цена от этого не возрастает.
- Операции разрешено применять только к исходным символам строки; запрещено применять операции к символам, которые уже были получены в результате предыдущих операций. Иными словами, строка преобразуется слева направо.

Параметры входных данных

В первой строке входных данных записаны три целых числа: sc , ci , cd — стоимости операций замены, вставки и удаления одного символа соответственно.

Во второй строке записана исходная строка A из строчных латинских букв, $1 \leq |A| \leq 2550$.

В третьей строке записана целевая строка B из строчных латинских букв, $1 \leq |B| \leq 2550$.

После вывода базового результата пользователь вводит номер расширяемой строки (1 или 2) и строку-продолжение.

Параметры выходных данных

Требуется вывести:

- таблицу динамического программирования со стоимостями (после базового вычисления);
- таблицу выбранных операций;
- последовательность операций, приводящую строку A к строке B ;
- итоговую минимальную стоимость преобразования;
- после расширения — обновлённую таблицу и новое минимальное расстояние.

Описание алгоритма

1. Идея динамического программирования

Для решения базовой задачи используется метод динамического программирования. Вводится состояние:

$dp[i][j]$ — минимальная стоимость преобразования первых i символов строки A в первые j символов строки B .

Индексы i и j принимают значения от 0 до n и от 0 до m соответственно, где $n = |A|$, $m = |B|$.

Полная таблица dp размером $(n+1) \times (m+1)$ сохраняется в памяти для последующего инкрементального расширения.

2. Базовые случаи

Если строка A пуста, то получить префикс строки B можно только вставками:

$$dp[0][j] = j \cdot ci, \text{ для } j = 0..m.$$

Если строка B пуста, то префикс строки A можно преобразовать только удалениями:

$$dp[i][0] = i \cdot cd, \text{ для } i = 0..n.$$

3. Рекуррентная формула (базовая часть)

При вычислении $dp[i][j]$ рассматриваются три варианта перехода:

Совпадение или замена. Сравниваются символы $A[i-1]$ и $B[j-1]$. Если они равны, выполняется сохранение и стоимость не увеличивается. Иначе выполняется замена со стоимостью cr :

$$dp[i-1][j-1] + (0, \text{ если } A[i-1] == B[j-1], \text{ иначе } cr).$$

Удаление одного символа. Удаляется символ $A[i-1]$, стоимость операции cd :

$$dp[i-1][j] + cd.$$

Вставка одного символа. Вставляется символ $B[j-1]$, стоимость операции ci :

$$dp[i][j-1] + ci.$$

Из всех допустимых вариантов выбирается минимальный. В таблицу операций $op[i][j]$ записывается код выбранного перехода: **M** — совпадение, **R** — замена, **D** — удаление, **I** — вставка.

4. Инкрементальное расширение строки (вариант 10)

После заполнения базовой таблицы dp и получения расстояния $dp[n][m]$ алгоритм переходит к этапу расширения.

Случай 1: расширяется строка **A** на k символов.

Новая строка $A' = A + extension$, её длина $n' = n + k$. Таблица dp расширяется на k новых строк (индексы $n+1 \dots n'$). Для этого к списку строк dp добавляются k новых строк, инициализированных нулями, **без копирования уже существующих строк** — это обеспечивает требуемую асимптотику.

Для каждой новой строки $i \in [n+1, n']$:

- Заполняется базовый случай: $dp[i][0] = i \cdot cd$.
- Для каждого $j \in [1, m]$ вычисляется $dp[i][j]$ по стандартной рекуррентной формуле, используя символы $A'[i-1]$ и $B[j-1]$.

Новое расстояние равно $dp[n'][m]$.

Сложность: $O(k \cdot m)$ — зависит только от длины добавления k и длины другой строки m , не зависит от исходной длины n .

Случай 2: расширяется строка **B** на k символов.

Новая строка $B' = B + extension$, её длина $m' = m + k$. К каждой существующей строке таблицы dp добавляются k новых элементов (нулей) — **без создания новых списков**, только мутация на месте через `extend`.

Для каждого нового столбца $j \in [m+1, m']$:

- Заполняется базовый случай: $dp[0][j] = j \cdot ci$.

- Для каждого $i \in [1, n]$ вычисляется $dp[i][j]$ по стандартной рекуррентной формуле, используя символы $A[i-1]$ и $B'[j-1]$.

Новое расстояние равно $dp[n][m']$.

Сложность: $O(n \cdot k)$ — зависит только от длины другой строки n и длины добавления k , не зависит от исходной длины m .

5. Восстановление последовательности операций

После заполнения таблиц dp и or выполняется обратный проход от клетки $[n][m]$ к клетке $[0][0]$. На каждом шаге анализируется код операции в $or[i][j]$:

- **M** — символы совпали, переход в $[i-1][j-1]$;
- **R** — замена, переход в $[i-1][j-1]$;
- **D** — удаление одного символа, переход в $[i-1][j]$;
- **I** — вставка, переход в $[i][j-1]$.

Полученная последовательность операций разворачивается и выводится как предписание.

При инкрементальном расширении восстановление предписания выполняется заново по обновлённой таблице, начиная с клетки $[n'][m]$ или $[n][m']$ в зависимости от того, какая строка расширялась.

6. Оценка сложности

Временная сложность базовой части. Таблица динамического программирования имеет размер $(n+1) \times (m+1)$. Каждая ячейка заполняется за константное число действий: проверка трёх переходов и выбор минимума. Следовательно, время работы базового алгоритма составляет $O(n \cdot m)$.

Временная сложность расширения. При добавлении k символов к одной из строк пересчитываются только новые строки или

столбцы таблицы. Сложность составляет $O(k \cdot L)$, где L — длина другой (не расширяемой) строки. Это удовлетворяет требованию варианта: сложность не зависит от изначальной длины расширенной строки.

Сложность по памяти. В реализации хранятся полные таблицы dp и or , поэтому память составляет $O(n \cdot m)$. Это сделано намеренно: полные таблицы необходимы для наглядного вывода промежуточных вычислений, для восстановления последовательности операций и для инкрементального расширения без пересчёта с нуля. При необходимости память базовой части можно сократить до $O(m)$, храня только две строки таблицы, однако тогда инкрементальное расширение потребует отдельного прохода или хранения дополнительной информации.

Тестирование:

Входные данные | Выходные данные | Комментарий

1 1 1; abc; abc; 1; def	Базовое: 0 (М М М) / Новое: 3 (М М М D D D)	Расширение первой строки, полное совпадение → удаление добавленных символов
1 1 1; abc; abc; 2; def	Базовое: 0 (М М М) / Новое: 3 (М М М I I I)	Расширение второй строки, полное совпадение → вставка добавленных символов
1 1 1; abc; xyz; 1; def	Базовое: 3 (R R R) /	Полное

	Новое: 6 (R R R D D D)	несовпадение, расширение первой → замены + удаления
1 1 1; abc; xyz; 2; uvw	Базовое: 3 (R R R) / Новое: 6 (R R R I I I)	Полное несовпадение, расширение второй → замены + вставки
1 1 1; abc; abc; 1; d	Базовое: 0 (M M M) / Новое: 1 (M M M D)	Расширение первой на 1 символ, совпадение → одно удаление
1 1 1; abc; abc; 2; d	Базовое: 0 (M M M) / Новое: 1 (M M M I)	Расширение второй на 1 символ, совпадение → одна вставка
1 1 1; abc; xyz; 1; a	Базовое: 3 (R R R) / Новое: 4 (R R R D)	Расширение первой на 1 символ, несовпадение → замены + удаление
1 1 1; abc; xyz; 2; a	Базовое: 3 (R R R) / Новое: 4 (R R R I)	Расширение второй на 1 символ, несовпадение → замены + вставка
1 1 1; (пусто); abc; 1; def	Базовое: 3 (I I I) / Новое: 6 (I I I I I I)	Пустая первая строка, расширение на непустую → все вставки

1 1 1; abc; (пусто); 2; def	Базовое: 3 (D D D) / Новое: 6 (D D D I I I)	Пустая вторая строка, расширение на непустую → удаления + вставки
1 1 1; (пусто); (пусто); 1; abc	Базовое: 0 / Новое: 3 (D D D)	Обе пустые, расширение первой → удаления
1 1 1; (пусто); (пусто); 2; abc	Базовое: 0 / Новое: 3 (I I I)	Обе пустые, расширение второй → вставки
10 1 1; abc; xyz; 1; def	Базовое: 3 (D D D I I I) / Новое: 6 (D D D I I I I I)	Дорогие замены: выгоднее удалить и вставить
1 10 1; abc; xyz; 2; def	Базовое: 3 (R R R) / Новое: 6 (R R R D D D)	Дорогие вставки: выгоднее удалить добавленное
1 1 10; abc; xyz; 1; def	Базовое: 3 (R R R) / Новое: 6 (R R R I I I)	Дорогие удаления: выгоднее вставить добавленное
1 1 1; abc; abcdef; 1; def	Базовое: 3 (I I I) / Новое: 0 (M M M M M M)	Расширение делает строки идентичными
1 1 1; abcdef; abc; 2; def	Базовое: 3 (D D D) / Новое: 0 (M M M M M M)	Расширение второй делает строки идентичными
1 1 1; abc; abcxyz; 1; xyz	Базовое: 3 (I I I) / Новое: 0 (M M M M M M)	Частичное улучшение до полного совпадения

1 1 1; aaa; aaa; 1; aaa	Базовое: 0 (М М М) / Новое: 3 (М М М D D D)	Расширение одинаковыми символами
1 1 1; abc; xyz; 1; aaa	Базовое: 3 (R R R) / Новое: 6 (R R R D D D)	Расширение на символы, которые всё равно удаляются
1 1 1; аааааааааа; аааааааааа; 1; аааааааааа	Базовое: 0 (10×М) / Новое: 10 (10×М + 10×D)	Длинные строки, расширение первой
1 1 1; аааааааааа; аааааааааа; 2; аааааааааа	Базовое: 0 (10×М) / Новое: 10 (10×М + 10×I)	Длинные строки, расширение второй
1 1 1; abc; def; 1; ghi	Базовое: 3 (R R R) / Новое: 6 (R R R D D D)	Смешанные операции при расширении
1 1 1; abc; abc; 1; abc	Базовое: 0 (М М М) / Новое: 3 (М М М D D D)	Расширение на копию строки
1 1 1; abc; xyz; 2; abc	Базовое: 3 (R R R) / Новое: 3 (R R R М М М)	Расширение с частичным совпадением в конце
1 1 1; abc; xyz; 1; (пусто)	Базовое: 3 (R R R) / Новое: 3 (R R R)	Расширение на пустую строку — результат не меняется
1 1 1; a; b; 1; c	Базовое: 1 (R) / Новое: 2 (R D)	Односимвольные строки, расширение первой

1 1 1; a; b; 2; c	Базовое: 1 (R) / Новое: 2 (R I)	Односимвольные строки, расширение второй
1 1 1; ab; abcde; 1; cde	Базовое: 3 (I I I) / Новое: 0 (M M M M M)	Расширение до полной длины другой строки
1 1 1; abcd; ab; 1; cd	Базовое: 2 (D D) / Новое: 2 (D D M M)	Расширение первой совпадающим суффиксом
1 1 1; ab; abcd; 2; cd	Базовое: 2 (I I) / Новое: 2 (M M I I)	Расширение второй совпадающим суффиксом
1 1 1; a; a; 1; a	Базовое: 0 (M) / Новое: 1 (M D)	Минимальный случай, расширение на тот же символ
1 1 1; a; a; 2; a	Базовое: 0 (M) / Новое: 1 (M I)	Минимальный случай, расширение второй на тот же символ
1 1 1; ab; cd; 1; ab	Базовое: 2 (R R) / Новое: 2 (R R M M)	Расширение первой совпадающим префиксом второй
1 1 1; cd; ab; 2; cd	Базовое: 2 (R R) / Новое: 2 (M M R R)	Расширение второй совпадающим префиксом первой
1 1 1; abc; def; 1; abc	Базовое: 3 (R R R) / Новое: 3 (R R R M M M)	Расширение первой на её же начало

1 1 1; abc; def; 2; def	Базовое: 3 (R R R) / Новое: 3 (M M M R R R)	Расширение второй на её же начало
1 1 1; abc; abc; 1; xyz	Базовое: 0 (M M M) / Новое: 3 (M M M R R R)	Расширение первой на несовпадающие символы → замены
1 1 1; abc; abc; 2; xyz	Базовое: 0 (M M M) / Новое: 3 (M M M R R R)	Расширение второй на несовпадающие символы → замены
1 1 1; abc; def; 1; def	Базовое: 3 (R R R) / Новое: 3 (R R R M M M)	Расширение первой на целевую строку → совпадение хвоста
1 1 1; def; abc; 2; def	Базовое: 3 (R R R) / Новое: 3 (M M M R R R)	Расширение второй на исходную строку → совпадение головы
1 1 1; abcd; abcd; 1; abcd	Базовое: 0 (M M M M) / Новое: 4 (M M M M D D D D)	Расширение на полную копию строки
1 1 1; abcd; abcd; 2; abcd	Базовое: 0 (M M M M) / Новое: 4 (M M M M I I I I)	Расширение второй на полную копию строки
1 1 1; a; (пусто); 1; a	Базовое: 1 (D) / Новое: 0 (M)	Расширение первой до совпадения со второй
1 1 1; (пусто); a; 2; a	Базовое: 1 (I) / Новое: 0 (M)	Расширение второй до совпадения с первой

1 1 1; ab; ba; 1; ab	Базовое: 2 (R R) / Новое: 4 (R R D D)	Расширение первой на исходную строку
1 1 1; ab; ba; 2; ab	Базовое: 2 (R R) / Новое: 4 (R R I I)	Расширение второй на исходную строку
1 1 1; abc; cba; 1; abc	Базовое: 2 (R R) / Новое: 5 (R R D D D)	Расширение первой на исходную строку
1 1 1; abc; cba; 2; abc	Базовое: 2 (R R) / Новое: 5 (R R I I I)	Расширение второй на исходную строку

Вывод

В ходе выполнения лабораторной работы был реализован алгоритм Вагнера-Фишера для вычисления редакционного расстояния с произвольными положительными стоимостями операций замены, вставки и удаления. Программа корректно строит матрицу динамического программирования, восстанавливает оптимальное редакционное предписание и визуализирует промежуточные результаты вычислений.

Специфика варианта 10 потребовала реализации механизма инкрементального обновления решения. После базового расчёта пользователь может расширить любую из строк, при этом новое расстояние вычисляется путём дозаполнения существующей таблицы DP, а не её полного пересчёта. Для обеспечения требуемой асимптотической сложности (независимости от исходной длины расширяемой строки) использована мутация массива на месте: новые строки или столбцы добавляются через `extend` без копирования ранее вычисленных данных. Это гарантирует временную сложность $O(k \cdot L)$ на этапе расширения, где k — длина добавленного фрагмента, а L — длина неизменяемой строки.

Реализация прошла проверку на широком наборе тестов, включая граничные случаи (пустые строки, расширение на один символ, нулевое изменение расстояния). Особое внимание уделено корректности границ циклов при дозаполнении матрицы, что исключает один за один ошибки и гарантирует учёт всех символов расширения. Таким образом, все требования индивидуального задания выполнены: обеспечена как функциональная корректность, так и заявленная эффективность алгоритма.